COL7524 Advanced Networks
July-December 2025

Assignment 2

Submitted by:
Nizaul Rahman | Entry number: 2025MCS2099
Arunangshu Pal | Entry number: 2025CSZ8238

Part A

For this assignment, a multithreaded client-server system was implemented where client requests are served by the server by three scheduling policies, namely First Come First Serve (FCFS), Shortest Job First (SJF), and Round Robin (RR). The server is programmed to record timestamps of the requests from clients to calculate metrics like waiting time, response time, and throughput to evaluate performance of each scheduling policy.
The server program and the client program read a json file called config.json to set the following parameters: server IP address, server port number, number of server threads (=4), number of client threads (=8).
Therefore, the server starts by creating 4 threads, and then each of the threads serves the incoming client requests by following one of the scheduling policies. The detailed experimental setup is given below. The server and client are run with the parameters mentioned below.

Experimental Setup

Server Configuration:
- Threads: 4
- Base directory: ./server_files
- Packetization parameter (--p): 10 (lines per packet)

Client Configuration:
- Threads: 8 concurrent clients
- Each client performs 10 operations (mix of PUT and GET)

Workload:
- Sample text files of varying sizes (small files for fast benchmarking)
- Each client thread randomly selects files for PUT and GET

Command-line Execution:

```
./server --sched fcfs --file ./server_files --p 10
./client BENCH ./client_files 10
```

Repeat for --sched sjf and --sched rr --quantum 5

Metrics Collected

i) Arrival Time: Timestamp when request arrived at the server
ii) Start Time: Timestamp when server started processing the request
iii) Finish Time: Timestamp when request finished processing
iv) Waiting Time: Start – Arrival
v) Response Time: Finish – Arrival
vi) Throughput: Requests per second over sliding 1-second window

All metrics are logged to server_log.csv and later processed using process_logs.py.

Graphs

The following graphs were generated by the script “process_logs.py”.

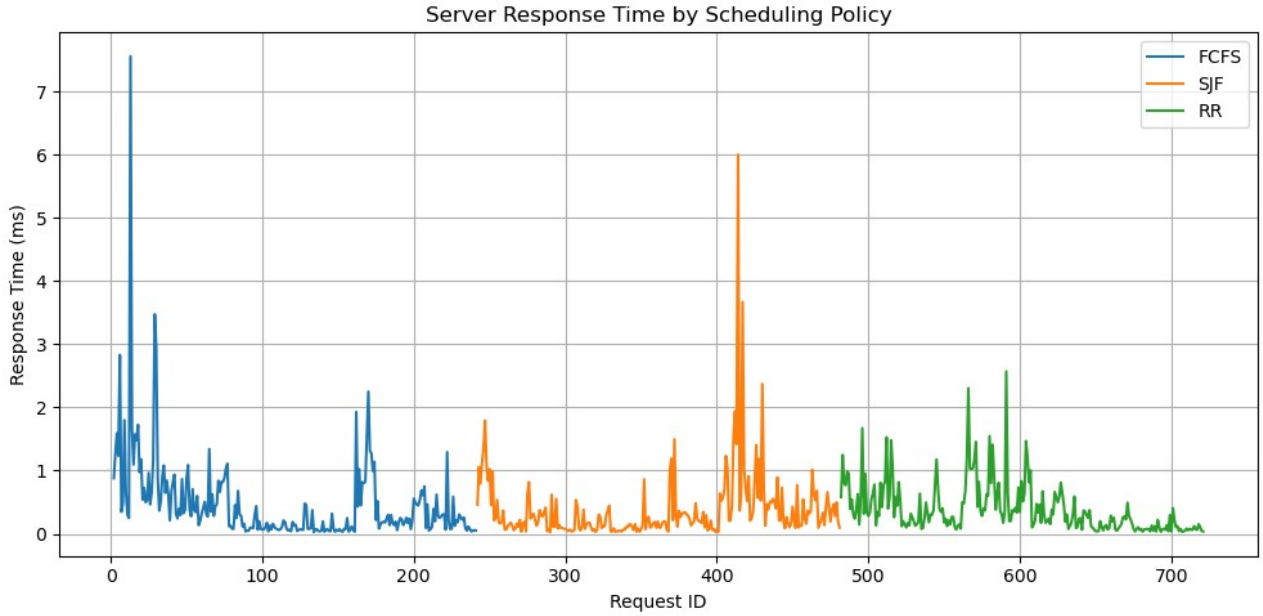

Figure 1: Response Time

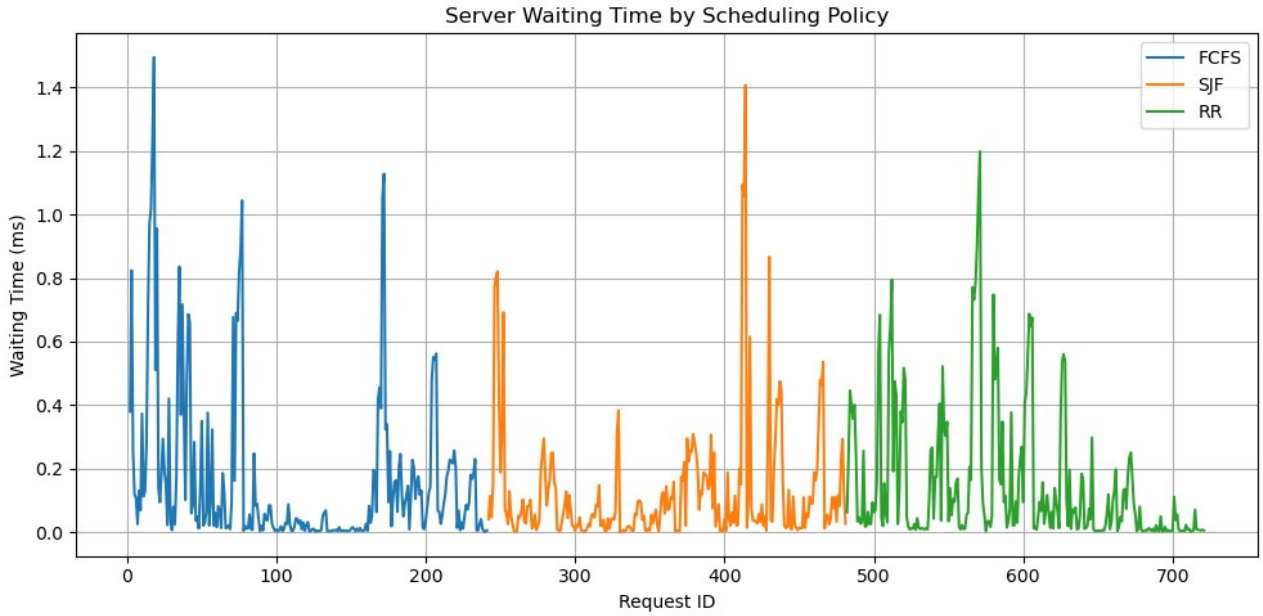

Figure 2: Waiting Time

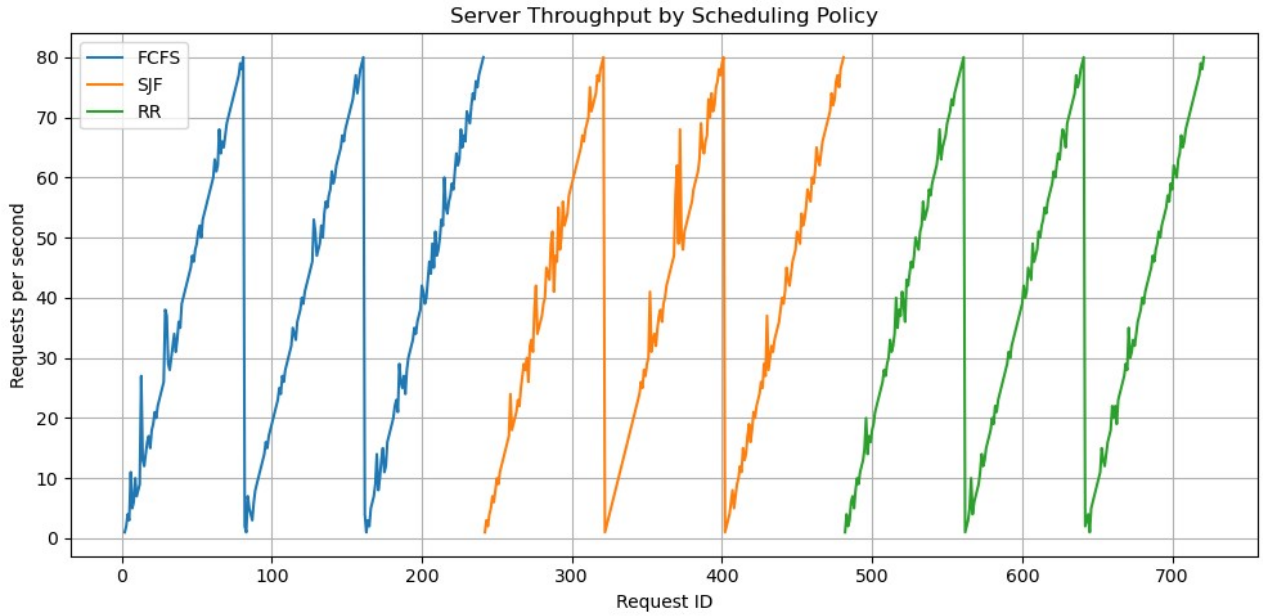

Figure 3: Throughput

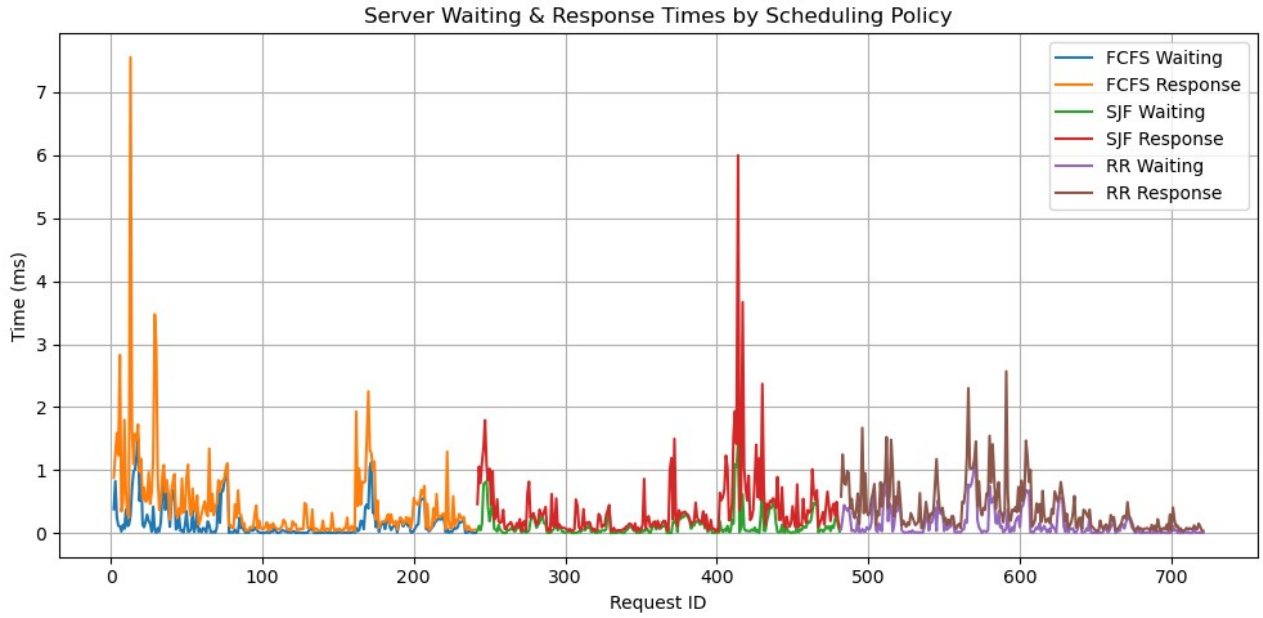

Figure 4: Overall comparison of performance

```
arunangshu@ESLab:~/MyPrograms/ASSIGN2$ python3 process_logs.py

Policy: FCFS
Total requests: 240
Average waiting time: 0.164 ms
Average response time: 0.464 ms
Average throughput: 40.500 req/sec

Policy: SJF
Total requests: 240
Average waiting time: 0.126 ms
Average response time: 0.380 ms
Average throughput: 40.500 req/sec

Policy: RR
Total requests: 240
Average waiting time: 0.149 ms
Average response time: 0.387 ms
Average throughput: 40.504 req/sec
arunangshu@ESLab:~/MyPrograms/ASSIGN2$
```

Figure 5: Running the script

```
arunangshu@ESLab:~/MyPrograms/ASSIGN2$ ./server --sched sjf --file ./server_files --p 2
[Server] listening 127.0.0.1:9000 sched=sjf p=2 threads=4
[+] New connection from 127.0.0.1:45696
[Request #0] ARRIVED: GET tiger_new.gdt
[Request #0] START processing: GET tiger_new.gdt | Waiting Time: 0.057 ms
[Request #0] FINISHED GET tiger_new.gdt | Response Time: 0.144 ms (Waiting: 0.057 ms)
[+] New connection from 127.0.0.1:58112
[Request #1] ARRIVED: PUT tiger_zinda_hai_3.gdt
[Request #1] START processing: PUT tiger_zinda_hai_3.gdt | Waiting Time: 0.091 ms
[Request #1] FINISHED PUT tiger_zinda_hai_3.gdt | Response Time: 0.569 ms (Waiting: 0.091 ms)
[+] New connection from 127.0.0.1:58126
[Request #2] ARRIVED: GET tiger_new.gdt
[Request #2] START processing: GET tiger_new.gdt | Waiting Time: 0.044 ms
[Request #2] FINISHED GET tiger_new.gdt | Response Time: 0.136 ms (Waiting: 0.044 ms)
```

Figure 6: Running the server

```
arunangshu@ESLab:~/MyPrograms/ASSIGN2$ ./client GET tiger_new.gdt ./cl_dir/newyy.gdt
[T-1] [GET] Received "./cl_dir/newyy.gdt" (17 bytes)
arunangshu@ESLab:~/MyPrograms/ASSIGN2$ ./client PUT ./client_files/tiger.gdt tiger_zinda_hai_3.gdt
[T-1] [PUT] Server: OK
arunangshu@ESLab:~/MyPrograms/ASSIGN2$ ./client GET tiger_new.gdt ./cl_dir
[T-1] [GET] Received "./cl_dir/tiger_new.gdt" (17 bytes)
arunangshu@ESLab:~/MyPrograms/ASSIGN2$ ./client PUT ./client_files/tiger.gdt tiger_zinda_hai_3.gdt
[T-1] [PUT] Server: OK
arunangshu@ESLab:~/MyPrograms/ASSIGN2$ ./client GET tiger_new.gdt ./cl_dir/newyy.gdt
[T-1] [GET] Received "./cl_dir/newyy.gdt" (17 bytes)
arunangshu@ESLab:~/MyPrograms/ASSIGN2$
```

Figure 6: Running the client

Results

Waiting and Response Times:

Observation:

- SJF yields the lowest average waiting time due to prioritization of small jobs.
- FCFS shows higher waiting for later requests if large requests arrive first.
- RR provides a balance between fairness and performance, but slightly higher average response time compared to SJF.

Throughput:

Observation:

- SJF and FCFS have similar peak throughput for small workloads.
- RR maintains consistent throughput due to time slicing, preventing long waits for any request.

Analysis and Trade-offs:

Policy	Pros	Cons
FCFS	Simple, easy to implement	Can have high waiting times for large requests
SJF	Minimizes average waiting time	Starvation of large requests possible
RR	Fair, avoids starvation	Slightly higher average response time due to context switching

Effect of packetization (--p):

- Higher p reduces the number of send() calls per file, improving throughput but may increase individual request latency.

Server/Client thread scaling:

- Increasing server threads improves concurrency and reduces waiting time under heavy client load.
- Increasing client threads simulates higher load and reveals scheduler efficiency.

Conclusion

- The multithreaded server handles concurrent requests with all three scheduling policies.
- Per-request logging enables detailed analysis of waiting times, response times, and throughput.
- SJF is best for minimizing waiting times, RR ensures fairness, and FCFS is the simplest but may penalize large jobs.
- The experimental setup demonstrates how different scheduling policies affect performance metrics under varying client loads and packetization parameters.

Submitted Files

- Source Code: server.cpp, client.cpp, Makefile, run_exps.sh, process_logs.py
- Logs: logs/fcfs_log.csv, logs/sjf_log.csv, logs/rr_log.csv
- Plots: waiting_response_comparison.png, throughput_comparison.png
- Sample Files: ./client_files/*
- Scripts: run_exps.sh to automate benchmarks